DzVisionAI Frontend: ALPR Integration & System Overview

Technologies & Key Dependencies

- **Framework:** Next.js (React-based, App Router)
- **Language:** TypeScript
- **Styling:** Tailwind CSS, custom CSS
- **State Management:** React hooks, Zustand
- **UI Components:** Custom React components, Headless UI, React Icons
- **HTTP Client:** Axios
- **Data Handling:** xlsx for CSV export
- **Image Handling:** next/image for optimized images
- **Notifications:** Custom notification stack
- **Authentication:** JWT (JSON Web Token) via cookies and Axios interceptors
- **Animation/UX:** AOS (Animate On Scroll)
- **Date Utilities:** date-fns
- **Charts:** chart.js

> **Note:** While the backend may use socket.io or client-socket-io, the frontend codebase (as of this report) does not directly use socket.io-client or real-time sockets. JWT is used for authentication and API security.

Project Structure (Relevant to ALPR)

```
src/
  app/
    Dashboard.tsx          # Main dashboard, ALPR upload & results
    services/
      videoService.ts      # Handles ALPR API calls (upload/process)
      tabService.ts        # Fetches plates/events/vehicles
      blacklist.ts         # Blacklist management (API)
    black-lists/           # Blacklist UI
    ui/
      features.tsx         # System features (LAPI, etc.)
      stats.tsx            # AI performance metrics
      tabs.tsx             # System component descriptions
      target.tsx           # Security features (ALPR, blacklist)
  components/
    notification.tsx       # Notification system
```

ALPR (Automatic License Plate Recognition) AI Integration

1. **User Flow**

- **Upload:** Users can upload images or videos from the dashboard.
- **Processing:** Uploaded media is sent to the backend ALPR API for processing.
- **Results:** The AI-detected license plate(s) and vehicle info are displayed, including images and metadata.
- **Blacklist Check:** Detected plates are checked against a blacklist; alerts are generated for matches.

2. **Key Components & Services**

Dashboard (`src/app/Dashboard.tsx`)

- **Upload Section:** Lets users upload images/videos.
- **Preview & Process:** Shows a preview and triggers processing via API.
- **Results Display:** Shows detected plate number, plate image, vehicle image, detection time, etc.
- **Pagination & Tabs:** Allows browsing of plates, events, vehicles, drivers, and cameras.

ALPR API Integration (`src/app/services/videoService.ts`)

Handles all communication with the backend ALPR API:

```
export const videoService = {
  async uploadImage(file: File) { ... },   // POST /video/upload_image
  async uploadVideo(file: File) { ... },   // POST /video/upload
  async processImage(filename: string) { ... }, // POST
/video/process_image/:filename
  async processVideo(filename: string) { ... }, // POST
/video/process/:filename
};
```

- **Upload:** Sends media as `multipart/form-data`.
- **Process:** Triggers AI inference on the backend; receives detection results.

Tab Data Fetching (`src/app/services/tabService.ts`)

- Fetches paginated lists of plates, events, vehicles, etc., for dashboard tabs.
- Example: `getLicensePlates(page, limit)` fetches recognized plates.

Blacklist Management (`src/app/services/blacklist.ts` & `black-lists/page.tsx`)

- **CRUD operations** for blacklisted plates.
- **Integration:** When a plate is detected, its number is checked against this list for alerts.

Notification System (`src/components/notification.tsx`)

- Provides user feedback for uploads, processing, errors, and blacklist alerts.

UI & User Experience

- **Modern, Responsive UI:** Built with Tailwind CSS and custom components.
- **Dark Mode:** Theme toggling based on user/system preference.
- **Real-Time Feedback:** Notifications for all major actions (upload, process, errors).
- **Performance Metrics:** Stats page shows ALPR accuracy, face detection accuracy, and response time.
- **System Overview:** Features and tabs pages describe all AI models and system components.

ALPR Data Flow

1. **User uploads image/video** via dashboard.
2. **Frontend calls** `videoService.uploadImage` or `uploadVideo`.
3. **Backend returns** a filename and preview URL.
4. **User triggers processing** (`handleProcessUpload`), which calls `videoService.processImage` or `processVideo`.
5. **Backend ALPR AI analyzes** the media, returns detection results:
    - Plate number, plate image, vehicle image, detection time, etc.
6. **Frontend displays results** and checks if the plate is in the blacklist.
7. **If blacklisted:** Notification/alert is shown.

Other System Components (Brief)

- **Events Tab:** Shows security events (e.g., detections, alerts).
- **Vehicles/Drivers/Cameras Tabs:** Manage and view related data.
- **User Management:** Admins can manage users and permissions.
- **Analytics:** Download CSV reports, view system stats.

Example: ALPR Integration Code Snippet

```
// Dashboard.tsx (simplified)
const handleFileChange = async (e, type) => {
  // Upload image/video
  const data = type === 'image'
    ? await videoService.uploadImage(file)
    : await videoService.uploadVideo(file);
  setUploadedFilename(data.filename);
  setUploadPreview(data.image_signed_url || data.thumbnail_signed_url);
};

const handleProcessUpload = async () => {
  // Process uploaded file
  const data = uploadType === 'image'
    ? await videoService.processImage(uploadedFilename)
    : await videoService.processVideo(uploadedFilename);
  setProcessResult(data);
  // Show notification based on result
};
```

AI Features Highlighted in UI

- **License Plate Recognition (LAPI):** Real-time, high-accuracy detection.
- **Blacklist Vehicle Detection:** Immediate alerts for unauthorized vehicles.
- **Performance Metrics:** 99.8% ALPR accuracy, 50ms response time (as shown in UI).

Summary

The DzVisionAI frontend is a modern, robust React/Next.js application that provides a seamless interface for interacting with advanced AI-powered surveillance features, with a strong focus on ALPR. The integration with the backend ALPR API is clean and modular, with clear separation of concerns between UI, services, and state management. The system is designed for real-time operation, user feedback, and high security, making it suitable for military-grade surveillance deployments.